Практическая работа № 1

Разработка перечня артефактов и протоколов проекта

1. Цель работы

Научиться определять, какие артефакты и протоколы необходимы для конкретного программного проекта, обосновывать их выбор и оформлять в виде структурированного перечня.

2. Выбор и описание проекта

Название проекта: «TaskFlow» — веб-приложение с мобильным клиентом для управления задачами и проектами внутри команды (учебный проект).

Цель проекта: создать единую систему, в которой команда планирует работу, распределяет задачи между участниками, отслеживает их выполнение и анализирует загрузку, чтобы сократить потери времени на переписку и уточнения.

Целевая аудитория: небольшие и средние команды разработки, студенческие проектные группы, руководители проектов, а также внештатные сотрудники и фрилансеры.

Ключевые функции:

- регистрация и авторизация пользователей с разграничением ролей (администратор, руководитель, исполнитель);
- создание проектов и задач с описанием, сроками, приоритетами и исполнителями;
- канбан-доска со статусами задач («К выполнению», «В работе», «На проверке», «Готово»);
- комментарии к задачам и вложение файлов;
- уведомления о новых задачах и приближении сроков (в приложении и по электронной почте);
- отчёты и аналитика: загрузка сотрудников, выполненные задачи, просроченные задачи;
- поиск и фильтрация задач по статусу, исполнителю, сроку и меткам.

Участники проекта:

- **Заказчик** — представитель организации, которая будет использовать систему;

- **Руководитель проекта** — планирование, контроль сроков и коммуникация с заказчиком;
- **Бизнес-аналитик** — сбор и документирование требований;
- **Архитектор** — проектирование структуры системы и базы данных;
- **Разработчики (frontend, backend, мобильный)** — реализация функций;
- **UX/UI-дизайнер** — проектирование интерфейса;
- **Тестировщики** — проверка качества системы;
- **DevOps-инженер** — сборка, развёртывание и сопровождение;
- **Технический писатель** — подготовка пользовательской документации.

3. Перечень артефактов

Перечень охватывает все этапы жизненного цикла проекта: от инициации до сопровождения. Всего определено 16 артефактов.

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап	Формат хранения
1	Паспорт (устав) проекта	Документ	Фиксирует цели, границы, сроки, бюджет и роли проекта	Руководитель проекта	Заказчик, вся команда	Инициация	Word / PDF, Confluence
2	Техническое задание (ТЗ)	Документ	Фиксирует требования заказчика к системе	Бизнес-аналитик	Вся команда, заказчик	Анализ требований	Word / PDF, Confluence
3	Спецификация требований и бэклог пользовательских историй	Документ / список	Детализирует функциональные и нефункциональные требования	Бизнес-аналитик, владелец продукта	Разработчики, тестировщики	Анализ требований	Jira (трекер задач)
4	Календарный план и реестр рисков	Документ / таблица	Определяет сроки, этапы, ответственных и возможные риски	Руководитель проекта	Команда, заказчик	Планирование	Excel, Jira
5	Макеты интерфейсов	Макет	Показывают внешний вид и логику экранов до разработки	UX/UI-дизайнер	Разработчики, заказчик	Проектирование	Figma
6	Архитектурное описание и диаграммы (UML, C4)	Схема	Описывает структуру системы, модули и связи между ними	Архитектор	Разработчики, DevOps	Проектирование	Draw.io, Confluence, репозиторий

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап	Формат хранения
7	Схема базы данных (ER-диаграмма)	Схема / БД	Определяет таблицы, связи и типы данных	Архитектор, backend-разработчик	Backend-разработчики, аналитик	Проектирование	PostgreSQL, файл .sql, PNG/PDF
8	Исходный код приложения	Код	Реализует функции системы	Frontend- и backend-разработчики	Разработчики, тестировщики, DevOps	Разработка	Git-репозиторий (GitHub / GitLab)
9	Документация API (OpenAPI/Swagger)	Документ	Описывает методы, форматы запросов и ответов серверной части	Backend-разработчик	Frontend-разработчики, мобильные разработчики, тестировщики	Разработка	YAML/JSON-файл в репозитории, Swagger UI
10	Конфигурационные файлы и скрипты CI/CD	Конфигурация	Автоматизируют сборку, проверку и развёртывание	DevOps-инженер	Разработчики, DevOps	Разработка, развёртывание	Файлы .yml, Dockerfile в репозитории
11	План тестирования и тест-кейсы	Тест-кейс	Определяют, что и как проверяется в системе	Тестировщик	Тестировщики, разработчики	Тестирование	TestRail / Excel
12	Отчёт о тестировании и баг-репорты	Отчёт	Фиксируют результаты проверок и найденные дефекты	Тестировщик	Руководитель проекта, разработчики	Тестирование	Jira, PDF
13	Инструкция по развёртыванию	Документ	Описывает установку и настройку системы на сервере	DevOps-инженер	Системный администратор, DevOps	Внедрение	Markdown (README) в репозитории
14	Руководство пользователя	Документ	Объясняет пользователям работу с системой	Технический писатель, аналитик	Пользователи, служба поддержки	Внедрение	PDF, раздел справки на сайте
15	Акт приёмки работ	Документ	Подтверждает соответствие результата требованиям ТЗ	Руководитель проекта	Заказчик, руководитель проекта	Приёмка и завершение	PDF с подписями
16	Журнал изменений релизов (Changelog)	Документ	Фиксирует, что изменилось в каждой версии	Разработчики, руководитель проекта	Команда, пользователи, поддержка	Сопровождение	Файл CHANGELOG.md в репозитории

4. Перечень протоколов

Протоколы определяют правила взаимодействия участников: как согласуются требования, проверяется код, фиксируются дефекты и принимается результат.

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
1	Протокол совещания	Фиксация обсуждаемых вопросов, решений и задач с ответственными	Вся команда; при необходимости заказчик	Еженедельно (стендапы — ежедневно)	Документ в Confluence
2	Протокол согласования требований	Порядок утверждения и подписания требований и ТЗ заказчиком	Аналитик, заказчик, руководитель проекта	По событию (при каждой версии требований)	Документ с подписью, письмо заказчика, запись в Jira
3	Протокол управления изменениями	Порядок подачи, оценки и утверждения изменений в требованиях	Заказчик, руководитель проекта, аналитик, разработчики	По событию (при запросе на изменение)	Запрос на изменение (Change Request) в Jira
4	Протокол код-ревью	Правила проверки кода перед слиянием в основную ветку	Автор кода, ревьюеры, ведущий разработчик	По событию (при каждом Pull Request)	Комментарии и одобрения в Pull Request
5	Протокол тестирования и учёта дефектов	Порядок выполнения тестов, оформления и закрытия багов	Тестировщики, разработчики, руководитель проекта	В каждой итерации и перед релизом	Отчёт и баг-репорты в Jira
6	Протокол выпуска релиза	Порядок подготовки, проверки и выкладки новой версии на сервер	DevOps, руководитель проекта, тестировщик	По событию (раз в 2 недели)	Чек-лист релиза, запись в Changelog
7	Протокол приёмки	Порядок передачи и принятия результата заказчиком	Заказчик, руководитель проекта, аналитик	Разово (по завершении этапов и проекта)	Акт приёмки с подписями сторон
8	Протокол обмена данными (API)	Форматы, частота и ответственность при обмене данными между клиентом, сервером и внешними сервисами	Backend- и frontend-разработчики, DevOps	Постоянно; пересматривается при изменении API	Документация OpenAPI в репозитории

5. Связь артефактов и протоколов

Артефакты — это результаты работы, а протоколы — правила, по которым эти результаты создаются, проверяются и утверждаются. Основные связи в проекте «TaskFlow»:

- **Протокол согласования требований и ТЗ.** Регулирует, как заказчик утверждает артефакт «Техническое задание». Без него ТЗ могло бы изменяться без ведома заказчика, и результат не совпал бы с его ожиданиями.
- **Протокол управления изменениями и спецификация требований.** Определяет, как новое требование попадает в бэклог и влияет на календарный план, макеты и тест-кейсы. Без протокола изменения вносились бы стихийно, а артефакты рассогласовались.
- **Протокол код-ревью и исходный код.** Любой фрагмент кода попадает в основную ветку репозитория только после проверки другим разработчиком. Это повышает качество кода и распространяет знания о нём внутри команды.
- **Протокол тестирования, тест-кейсы и отчёт о тестировании.** Тест-кейсы задают, что проверять, а протокол — как фиксировать результаты. Итогом становится отчёт и баг-репорты, на основании которых принимается решение о релизе.
- **Протокол обмена данными (API) и документация API.** Протокол задаёт форматы и правила обмена между клиентом и сервером, а документация OpenAPI фиксирует их в виде артефакта, которым пользуются frontend-разработчики и тестировщики.
- **Протокол приёмки и акт приёмки работ.** Протокол определяет порядок проверки результата заказчиком, а акт подтверждает его соответствие ТЗ и служит основанием для завершения этапа.

6. Обоснование выбора

6.1. Обязательные и специфичные артефакты

Обязательные для любого проекта: паспорт проекта, ТЗ и спецификация требований, календарный план и реестр рисков, исходный код, план тестирования и отчёт о тестировании, инструкция по развёртыванию, руководство пользователя, акт приёмки. Без них невозможно ни согласовать результат, ни воспроизвести его, ни передать заказчику.

Специфичные для «TaskFlow»: макеты интерфейсов (в системе много экранов: доска, карточки задач, отчёты), документация API (есть веб-клиент, мобильный клиент и сервер), схема базы данных (много связанных сущностей: пользователи, проекты, задачи, комментарии), конфигурации CI/CD (частые релизы) и журнал изменений.

6.2. Критичные протоколы

Для «TaskFlow» наиболее критичны протоколы согласования требований и управления изменениями, так как заказчик часто уточняет функции уже в ходе работы. Протокол код-ревью и протокол тестирования защищают качество при параллельной работе нескольких разработчиков.

Протокол обмена данными (API) необходим, потому что с одним сервером работают сразу два клиента. Протокол выпуска релиза снижает риск сбоев при выкладке, а протокол приёмки формально завершает работу.

6.3. Последствия отсутствия артефакта или протокола

Что отсутствует	Возможные последствия
Техническое задание / спецификация требований	Нет общего понимания, что нужно сделать; заказчик и команда по-разному понимают результат, растёт объём переделок.
Схема БД и документация API	Разработчики вынуждены угадывать структуру данных и формат запросов, возникают ошибки интеграции клиентов и сервера.
Тест-кейсы и отчёт о тестировании	Дефекты обнаруживаются уже у пользователей, нет оснований для решения о выпуске релиза.
Инструкция по развёртыванию	Развёртывание зависит от одного человека; при его уходе или сбое систему невозможно быстро восстановить.
Протокол согласования требований	Требования утверждаются устно и позже оспариваются, возникают конфликты с заказчиком.
Протокол управления изменениями	Изменения вносятся без оценки влияния на сроки и бюджет, план срывается.
Протокол код-ревью	В основную ветку попадают ошибки и нечитаемый код, ухудшается сопровождаемость.
Протокол приёмки / акт приёмки	Нет подтверждения, что работа выполнена; возможны споры об оплате и дополнительные требования после сдачи.

Итог: артефакты фиксируют результат работы, а протоколы обеспечивают единые правила его создания и утверждения. Совместно они делают проект управляемым, прозрачным и воспроизводимым.